

MEDICINE SHOP ASSISTANT & DOCTOR APPOINTMENT SYSTEM

PROJECT REPORT

Submitted by-

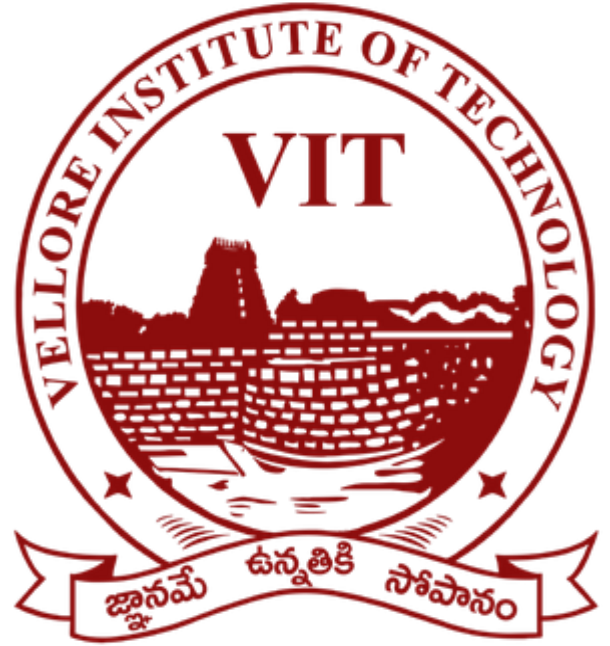
Name: Satwika

Registration number: 26BHI10057

Class/Sem : 1ST Sem

Subject: Introduction to problem solving

Academic Year: 2026-30



Submitted to-

Teacher/Professor: Mr. Darshan Teja

Department: SCAI

Institution: VIT Bhopal University

1. INTRODUCTION

Healthcare-related information systems can help organize basic patient information, medicine-related information, doctor details, appointments, and medicine-shop activities.

The **Medicine Shop Assistant & Doctor Appointment System** is a Python-based console application developed as an educational mini project. The application combines multiple functions into a single menu-driven system.

The project provides patient registration, patient details, symptom checking, general medicine information, severe-case detection, doctor information, appointment booking, medicine categories, first-aid information, medicine inventory, medicine billing, condition searching, and patient summaries.

The project is explicitly designed as an educational system and does not diagnose diseases or prescribe medicines.

2. OBJECTIVES

The main objectives of the project are:

1. To develop a menu-driven Python application.
2. To demonstrate the use of Python data structures.
3. To create a simple patient registration system.
4. To provide general information about common conditions.
5. To provide general information about medicines and supportive measures.
6. To identify predefined severe symptoms.
7. To display doctor information.
8. To demonstrate appointment booking.
9. To create a basic medicine inventory system.
10. To calculate medicine bills.
11. To demonstrate input validation and exception handling.

12. To understand how different programming concepts can be combined into one application.
-

3. PROBLEM STATEMENT

In a basic medicine-shop or healthcare environment, several activities may need to be handled, such as maintaining patient details, finding general information about common conditions, displaying available medicines, maintaining stock, calculating bills, and providing doctor information.

For a beginner-level programming project, these activities can be combined into a simple console-based application.

The objective is therefore to develop a Python program that demonstrates these functions using basic programming concepts without relying on external databases or complex frameworks.

4. SCOPE OF THE PROJECT

The scope of this project includes:

- Patient registration
- Patient information display
- Symptom keyword checking
- General condition information
- Medicine/supportive-measure information
- Severe symptom detection
- Doctor information
- Appointment booking
- Medicine categories
- First-aid information
- Medicine inventory
- Medicine billing
- Patient summary
- Menu-driven navigation

The current version is designed as an educational console application rather than a real healthcare-management system.

5. TECHNOLOGIES USED

5.1 Programming Language

Python

Python is used because it provides simple syntax and supports functions, dictionaries, lists, loops, conditional statements, exception handling, and string operations.

5.2 Data Structures

The project uses:

Dictionaries

Dictionaries are used to store:

- Medicine information
- Condition information
- Medicine inventory
- Patient information
- Doctor information

Lists

Lists are used for:

- Severe symptoms
- Doctors
- Medicine categories
- Medicine results

5.3 Programming Concepts

The project demonstrates:

- Variables
- Functions
- Lists
- Dictionaries

- Loops
- Conditional statements
- Exception handling
- String handling
- User input
- Menu-driven programming

These concepts are also listed in the program's own "About Project" section.

6. SYSTEM FEATURES

6.1 Patient Registration

The application starts by registering the patient.

The user enters:

- Name
- Age
- Gender
- Phone number
- Address
- Blood group
- Emergency contact

The program also validates the patient's age and handles invalid numerical input.

6.2 Patient Details

After registration, the system can display the patient's stored information.

This demonstrates how information can be stored in a Python dictionary and accessed using keys.

6.3 Symptom Checking

The user can enter symptoms into the program.

The system converts the entered text to lowercase and compares it against predefined conditions.

For recognized conditions, the system displays:

- General information
- Commonly used products/supportive measures

The program also includes a disclaimer stating that the results are general information and not a personalized prescription.

6.4 Severe Case Detection

A predefined list of severe symptoms is maintained in the program.

Examples include:

- Chest pain
- Difficulty breathing
- Severe bleeding
- Unconsciousness
- Seizure
- Stroke
- Severe abdominal pain
- Very high fever
- Severe headache
- Face swelling
- Blood vomiting
- Black stool
- Serious injury

If a matching severe symptom is found, the program displays an urgent medical-evaluation message rather than providing normal medicine information.

6.5 Medicine Information

The program contains a predefined medicine database.

Examples include information associated with:

- Fever
- Cough

- Cold
- Headache
- Stomach pain
- Leg pain
- Body pain
- Sore throat
- Acidity
- Indigestion
- Diarrhea
- Vomiting
- Constipation
- Allergy
- Skin itching
- Minor cuts
- Minor burns
- Muscle pain

The database provides general medicine or supportive-measure information rather than individualized prescriptions.

7. CONDITION INFORMATION

The project contains a condition-information dictionary.

For example, the program explains that fever can occur with infections and other conditions, while cough can occur due to infections, allergies, asthma, or irritation.

Similar general descriptions are provided for the other predefined conditions.

8. DOCTOR INFORMATION

The system contains a predefined doctor database.

Each doctor record contains:

- Name
- Speciality
- Phone number
- Available time
- Available days

The application can display these details to the user.

9. APPOINTMENT BOOKING

The appointment module allows the user to:

1. View available doctors.
2. Select a doctor.
3. Enter an appointment date.
4. Receive an appointment confirmation.

The confirmation displays the patient name, age, selected doctor, speciality, date, available time, and doctor phone number.

10. MEDICINE CATEGORIES

The project provides a list of medicine/product categories, including:

- Pain and fever relief
- Cold and cough products
- Digestive health products
- Allergy products
- Oral rehydration products
- First-aid products
- Skin-care products
- Throat-care products

This provides a simple way to organize the products conceptually.

11. FIRST-AID INFORMATION

The first-aid module provides basic educational information for:

1. Minor cuts
2. Minor burns
3. Fever
4. Dehydration
5. Serious injuries

For serious injuries, the system advises seeking professional medical assistance.

12. MEDICINE INVENTORY

The inventory module maintains medicine names along with their prices and available stock.

The current database includes items such as:

- Paracetamol
- ORS
- Cough Lozenges
- Saline Nasal Spray
- Antacid
- Bandage
- Antiseptic
- Calamine Lotion

Each item has a price and stock quantity.

13. BILLING SYSTEM

The billing module allows the user to select medicines and enter quantities.

The program:

1. Displays the inventory.
2. Accepts a medicine name.
3. Checks whether it exists.
4. Accepts the quantity.
5. Checks whether enough stock is available.
6. Calculates the item total.
7. Updates the inventory.
8. Calculates the grand total.
9. Displays the final bill.

The total is calculated using:

Item Total = Price × Quantity

The grand total is calculated by adding the totals of all purchased items.

14. PATIENT SUMMARY

The patient-summary module displays the registered patient's:

- Name
- Age
- Gender
- Phone
- Address
- Blood group
- Emergency contact

The project specifies that this information is maintained for the current project session.

15. SYSTEM WORKFLOW

The overall workflow of the application is:

START



Patient Registration



Main Menu



Select Operation



	Check Symptoms	
	Search Condition	
	View Patient Details	
	View Common Conditions	
	Medicine Categories	
	First-Aid Information	
	View Doctors	
	Book Appointment	
	View Medicine Inventory	
	Create Medicine Bill	
	Patient Summary	
	About Project	
	Exit	



Perform Selected Operation



Return to Main Menu



Exit



END

The main menu in the program contains 13 options corresponding to these operations.

16. IMPORTANT FUNCTIONS

The project is divided into multiple functions to make the program easier to understand and manage.

Important functions include:

`register_patient()`

`display_patient_details()`

`display_header()`

`display_conditions()`

`check_severe_case()`

`find_medicines()`

`find_condition_information()`

`symptom_check()`

`search_condition()`

`display_doctors()`

`book_appointment()`

`medicine_categories()`

`first_aid()`

`display_inventory()`

`create_bill()`

`patient_summary()`

`about_project()`

`main_menu()`

`start_program()`

This modular approach makes each major feature responsible for a specific task.

17. INPUT VALIDATION

Input validation is included in several parts of the program.

For example:

- Patient age must be within a specified range.
- Invalid age input is handled using exception handling.
- Invalid doctor numbers are rejected.
- Medicine quantities must be positive.
- Invalid quantities are handled.
- The system checks whether enough medicine stock is available.
- Invalid main-menu choices are rejected.

This improves the reliability of the program during normal use.

18. ALGORITHM

Algorithm for Patient Registration

1. Start
2. Ask for patient name
3. Ask for age
4. Validate age
5. Ask for gender
6. Ask for phone number
7. Ask for address
8. Ask for blood group
9. Ask for emergency contact
10. Store information in a dictionary
11. Display registration confirmation
12. Return patient information
13. End

Algorithm for Symptom Checking

1. Start
2. Accept symptoms from the user
3. Convert symptoms to lowercase
4. Check for severe symptoms
5. If severe symptom is found:
 - Display urgent medical-evaluation message
 - Optionally display doctor details
6. Otherwise:
 - Search the medicine database

Search condition information

Display matching information

7. End

Algorithm for Billing

1. Start

2. Display medicine inventory

3. Ask for medicine name

4. Check whether medicine exists

5. Ask for quantity

6. Validate quantity

7. Check available stock

8. Calculate item total

9. Reduce stock

10. Add item to cart

11. Repeat until user enters "done"

12. Calculate grand total

13. Display final bill

14. End

19. ADVANTAGES

The project has several advantages as an educational application:

1. Simple and easy-to-understand interface.
 2. Uses fundamental Python concepts.
 3. Combines multiple modules in one application.
 4. Demonstrates real-world programming logic.
 5. Includes input validation.
 6. Demonstrates inventory management.
 7. Demonstrates basic billing.
 8. Includes appointment-management logic.
 9. Provides a clear menu-driven workflow.
 10. Can serve as a foundation for a larger healthcare application.
-

20. LIMITATIONS

The current version has several limitations.

20.1 No Permanent Database

Patient and inventory information exists only during the current program session.

20.2 Console Interface

The project uses a text-based interface rather than a graphical or web interface.

20.3 Predefined Information

Conditions, medicines, doctors, and inventory items are stored directly in the Python source code.

20.4 Basic Symptom Matching

The symptom checker uses keyword matching rather than an advanced medical or machine-learning model.

20.5 No Real Appointment System

Appointment booking is simulated and is not connected to a real hospital or doctor's calendar.

20.6 Educational Use Only

The program does not provide medical diagnosis or personalized prescriptions.

21. FUTURE SCOPE

The project can be improved significantly in future versions.

Possible improvements include:

Database Integration

A database such as MySQL or SQLite could be added for permanent storage.

Graphical User Interface

A GUI could be developed using technologies such as Tkinter.

Web Application

The system could be converted into a web application.

Authentication

Separate login systems could be created for:

- Patients
- Doctors
- Pharmacists
- Administrators

Advanced Inventory Management

Additional inventory features could include:

- Expiry dates
- Low-stock alerts
- Supplier information
- Purchase records
- Sales history

Appointment Management

The appointment system could support:

- Time-slot selection

- Appointment cancellation
- Rescheduling
- Doctor availability
- Appointment history

Improved Search

The search system could support partial matches and more flexible symptom input.

22. CONCLUSION

The **Medicine Shop Assistant & Doctor Appointment System** demonstrates how fundamental Python programming concepts can be used to develop a practical menu-driven application.

The project combines patient registration, symptom checking, condition information, medicine information, severe-case detection, doctor information, appointment booking, first-aid information, inventory management, billing, and patient summaries.

Although the current version is a simple educational console application, its modular structure provides a foundation for future development into a database-driven or web-based healthcare-management application.

The project also demonstrates important programming concepts such as dictionaries, lists, functions, loops, conditional statements, string handling, exception handling, and user input.

Most importantly, the application is intended for **educational purposes only** and should not be treated as a replacement for professional medical diagnosis, prescription, or emergency medical care.

23. REFERENCES

The primary implementation reference for this report is the project's Python source code.

Project: Medicine Shop Assistant & Doctor Appointment System

Programming Language: Python

Project Type: Python Mini Project

Application Type: Console-based menu-driven application